ADR-006 — Linguagem e Stack de Desenvolvimento do Veridit

Status: Proposto | Data: 20/10/2026 | Autor(es): Equipe ___

Requisitos/restrições relacionados: REQ01–REQ15; ADR-001 (serviços implantados separadamente); ADR-005 (sessões e workers separados); ADR-007 (Playwright e streaming por WebSocket); restrições de contexto: equipe pequena, primeira versão, cronograma curto, código e instruções de execução no repositório.

Contexto

O estilo escolhido (ADR-001) exige múltiplos deployables — gateway, cinco serviços de domínio, workers de sessão e de pós-processamento — além do frontend web. O sistema precisa de: automação de navegador e streaming de tela em tempo real (ADR-007), SDK de pagamento Mercado Pago/PIX (REQ06), geração de relatório PDF (REQ14), ZIP (REQ15), SHA-256 (ADR-002) e Argon2id (ADR-004).

Domínio atual da equipe: _____ (preencher em sala). O domínio prévio da equipe é o critério de maior peso nesta decisão: se a equipe dominar outra stack com suporte oficial ao Playwright (Java, Python ou .NET), esta ADR deve ser reavaliada.

Decisão

Adotar TypeScript como linguagem única do sistema: backend (gateway e serviços de domínio) em Node.js LTS com NestJS; frontend web em React com TypeScript; workers de sessão e de pós-processamento em Node.js como processos separados. Contratos HTTP entre serviços em OpenAPI e contratos de mensagens em JSON Schema, com tipos gerados a partir das especificações.

Alternativas Consideradas

1. Java/Spring Boot — rejeitada: maturidade transacional e de segurança excelentes e Playwright oficial, mas manteria duas linguagens (backend Java, frontend TypeScript) e maior verbosidade para uma equipe pequena no cronograma.
2. Python (FastAPI) — rejeitada: produtiva e com Playwright oficial, mas também exige uma segunda linguagem no frontend.
3. Go — rejeitada: desempenho e concorrência excelentes, mas sem Playwright oficial (apenas port mantido pela comunidade) e ecossistema menor para PDF e pagamentos.

Justificativa

- Uma única linguagem em frontend, backend e workers elimina troca de contexto e permite compartilhar tipos de contratos (APIs e mensagens) — o fator técnico decisivo para equipe pequena e prazo curto, uma vez que Java, Python e .NET também têm Playwright oficial.
- O streaming de tela e eventos por WebSocket (ADR-007) é natural no modelo de I/O orientado a eventos do Node, e o mesmo código TypeScript roda no navegador do usuário e no servidor.
- O próprio Playwright gera o relatório em PDF a partir de um template HTML (page.pdf()), dispensando biblioteca adicional (REQ14); ZIP (archiver), SHA-256 (node:crypto), Argon2id (argon2) e SDK oficial do Mercado Pago estão disponíveis.
- NestJS organiza o código em módulos com injeção de dependência, facilitando replicar a mesma estrutura nos serviços (ADR-001) e implementar os adaptadores de pagamento (ADR-010).

Consequências

Positivas: um só runtime para construir, testar e implantar; contratos tipados de ponta a ponta; maior velocidade de evolução; menos dependências.

Negativas: tipagem menos rigorosa que Java/Go; runtime single-thread — tarefas de CPU pesada (vídeo, ZIP, hash) competiriam com requisições se executadas no mesmo processo; acoplamento à evolução do Node.

Mitigações: TypeScript em modo strict com ESLint; cargas de CPU apenas em workers separados (ADR-005, ADR-009); versão LTS do Node fixada na imagem de container (ADR-012).

Alterações em relação à versão anterior

- Domínio da equipe explicitado como critério principal, com campo a preencher em sala.
- Removida a comparação de desempenho de I/O com FastAPI (discutível); reconhecido que Playwright é oficial também em Java/Python/.NET — o diferencial é a linguagem única.
- Relatório PDF gerado pelo próprio Playwright; streaming por WebSocket (nova ADR-007).